



Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



"AWS S3: Bucket List: Master AWS S3. Build Big, Store Smarter."

What is AWS S3?

Amazon **Simple Storage Service (Amazon S3)** is an object storage service that offers industry-leading scalability, data availability, security, and performance. Customers of all sizes and industries can use Amazon S3 to store and protect any amount of data for a range of use cases, such as data lakes, websites, mobile applications, backup and restore, archive, enterprise applications, IoT devices, and big data analytics. Amazon S3 provides management features so that you can optimize, organize, and configure access to your data to meet your specific business, organizational, and compliance requirements.



Amazon S3

How it Works?

Amazon S3 is an object storage service that stores data as objects, hierarchical data, or tabular data within buckets. An *object* is a file and any metadata that describes the file. A *bucket* is a container for objects. To store your data in Amazon S3, you first create a bucket and specify a bucket name and AWS Region. Then, you upload your data to that bucket as objects in Amazon S3. Each object has a *key* (or *key name*), which is the unique identifier for the object within the bucket.

S3 provides features that you can configure to support your specific use case. For example, you can use S3 Versioning to keep multiple versions of an object in the same bucket, which allows you to restore objects that are accidentally deleted or overwritten. Buckets and the objects in them are private and can be accessed only if you explicitly grant access permissions. You can use bucket policies, AWS Identity and Access Management (IAM) policies, access control lists (ACLs), and S3 Access Points to manage access.

Before writing code or clicking buttons in the AWS console, you need to understand the fundamental building blocks of Amazon S3. These concepts govern how data is organized, secured, optimized, and paid for.

1. Data Organization & Identification

- **Bucket:** The fundamental container for data stored in S3. Every object lives inside a bucket. You can think of it like a top-level drive or a root folder, but with one critical rule: **its name must be globally unique** across all AWS accounts worldwide.
- **Object:** The fundamental entity stored in S3. Unlike a traditional computer that uses file storage, S3 uses **Object Storage**. An object consists of three things:
 - **Data (The File):** The actual byte stream (image, video, document).
 - **Metadata:** A set of name-value pairs that describe the object (e.g., file size, creation date, content type like image/jpeg).
 - **Key (The Unique ID):** The full path/name used to find the object.
- **Key & Key-Value Store:** S3 is fundamentally a massive key-value store. It does **not** have actual folders. Instead, it uses **prefixes** (slashes in the key name) to *mimic* a folder structure.





Cloud Computing & Dev-Ops



Transform Code into Cloud Solutions: Master AWS Today

- *Example:* If your object key is uploads/2026/avatar.jpg, the bucket doesn't actually contain a folder named uploads. The entire string is just the object's unique ID, but AWS management tools parse the / to display a familiar folder layout.

2. Security & Access Control

- **IAM (Identity and Access Management):** The AWS service used to manage permissions. You use IAM policies to grant specific users or services (like your Node.js application) permission to read or write to an S3 bucket.
- **Bucket Policy:** A security policy written in JSON attached **directly to the bucket**. It controls who (which AWS accounts or public users) can access the entire bucket or specific paths inside it.
- **Block Public Access (BPA):** A master switch at the bucket or account level. When turned on (which is the default), it completely blocks any configuration that would make your data public to the internet, acting as a safeguard against data leaks.
- **Encryption:** Securing data so unauthorized parties cannot read it. S3 handles this in two ways:
 - **Encryption in Transit:** Data is encrypted while moving between your computer and AWS (using HTTPS).
 - **Encryption at Rest:** Data is encrypted while sitting on the physical hard drives in AWS data centres. S3 automatically encrypts all new objects by default using **SSE-S3** (Amazon managed keys).

3. Data Lifecycle & Cost Optimization

- **Storage Classes:** S3 does not cost the same for all types of data. You choose a storage tier based on how often you need to access the file. The core classes are:
 - **S3 Standard:** For frequently accessed data (e.g., website images). High storage cost, zero retrieval cost.
 - **S3 Standard-IA (Infrequent Access):** For data accessed less often but needed immediately when requested (e.g., older user backups). Lower storage cost, but you pay a small fee per gigabyte retrieved.
 - **S3 Glacier (Flexible / Deep Archive):** For long-term data archiving (e.g., regulatory compliance logs). Extremely cheap storage, but retrieving the data can take anywhere from minutes to several hours.
- **Lifecycle Rules:** Automation rules you set up to manage your objects over time. For example, you can create a rule that says: *"30 days after a file is uploaded, automatically move it to S3 Infrequent Access, and after 365 days, delete it permanently."* This saves massive amounts of money automatically.
- **Versioning:** A feature that keeps multiple variants of an object in the same bucket. If you overwrite or delete a file with versioning turned on, AWS keeps the old version as a backup, protecting you against accidental deletions or application bugs.

4. Performance & Availability

- **Regions:** The physical geographic location (e.g., us-east-1 in Virginia, ap-south-1 in Mumbai) where AWS data centres host your bucket. You should pick a region closest to your users to minimize network latency.
- **Durability vs. Availability:** Two terms often confused in cloud computing:
 - **Durability (99.99999999% - "11 Nines"):** The probability that your data will not be lost or corrupted. S3 automatically replicates your objects across multiple physical data centres within a region, making data loss practically impossible.
 - **Availability (99.9%):** The probability that the S3 service is up, running, and accessible at any given moment.





Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



Creating a Bucket and Uploading a File

This walkthrough uses the web-based AWS Management Console. You will need an active AWS Account to follow along.

Step 1: Navigate to the S3 Dashboard

1. Log in to your AWS Management Console.
2. In the top search bar, type **S3** and select **S3 (Manage storage in the cloud)** from the dropdown.

Step 2: Create Your First Bucket

1. Click the orange **Create bucket** button.
2. **Bucket name:** Enter a unique, lowercase name (e.g., my-unique-app-backup-2026). If the name is already taken by someone else in the world, AWS will show an error.
3. **AWS Region:** Choose a geographic region closest to your users or infrastructure (e.g., *US East (N. Virginia)*) to reduce latency.
4. Leave **Block Public Access settings** turned **on** (recommended for safety).
5. Scroll to the bottom and click **Create bucket**.

Step 3: Upload an Object (File)

1. In your S3 dashboard, locate your newly created bucket from the list and **click its name**.
2. Click the orange **Upload** button.
3. Click **Add files** or simply drag and drop a file (like an image or a text file) from your local computer into the browser window.
4. Scroll down and click the blue **Upload** button at the bottom right. Your file is now safely stored in the cloud!





Cloud Computing & Dev-Ops



Transform Code into Cloud Solutions: Master AWS Today

Walkthrough of Simple Insurance static website and deploy it in two ways:

- Developer PC → IIS → Local website
- Same files → AWS S3 → Global/public website

Scenario

A small **Insurance Policy Calculator** using:

- HTML
- Tailwind CSS
- JavaScript
- No backend/database
- IIS for local hosting
- Amazon S3 for cloud hosting

The application will accept: Customer Name + Insurance Type + Coverage Amount and calculate a simple indicative premium in JavaScript.

Step 1: Create the Static Website

- Create a folder: InsuranceWebsite
- Inside folder
 - HTML File: index.html (case sensitive name)

```
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />

<title>SecureLife Insurance</title>

<!-- Tailwind CSS CDN -->
<script src="https://cdn.tailwindcss.com"></script>
</head>

<body class="bg-gray-100 min-h-screen">
<!-- Header -->
<header class="bg-blue-700 text-white">
<div class="max-w-6xl mx-auto px-6 py-5">
<h1 class="text-3xl font-bold">SecureLife Insurance</h1>

<p class="text-blue-100">Simple Insurance Premium Calculator</p>
</div>
</header>

<!-- Main -->
<main class="max-w-6xl mx-auto px-6 py-10">
<div class="grid md:grid-cols-2 gap-8">
```





Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



```
<!-- Information -->
<section class="bg-white rounded-lg shadow p-8">
<h2 class="text-2xl font-bold text-gray-800 mb-4">
Protect What Matters
</h2>

<p class="text-gray-600 mb-6">
Calculate an indicative insurance premium based on your selected
policy and coverage.
</p>

<div class="space-y-4">
<div class="border-l-4 border-blue-600 pl-4">
<h3 class="font-semibold">Life Insurance</h3>
<p class="text-sm text-gray-500">Protection for your family.</p>
</div>

<div class="border-l-4 border-green-600 pl-4">
<h3 class="font-semibold">Health Insurance</h3>
<p class="text-sm text-gray-500">
Healthcare coverage for you and your family.
</p>
</div>

<div class="border-l-4 border-purple-600 pl-4">
<h3 class="font-semibold">Vehicle Insurance</h3>
<p class="text-sm text-gray-500">Protection for your vehicle.</p>
</div>
</div>
</section>

<!-- Calculator -->
<section class="bg-white rounded-lg shadow p-8">
<h2 class="text-2xl font-bold text-gray-800 mb-6">
Premium Calculator
</h2>

<form id="premiumForm">
<!-- Customer -->
<div class="mb-4">
<label class="block font-medium mb-2"> Customer Name </label>

<input
type="text"
id="customerName"
class="w-full border rounded px-4 py-2"
placeholder="Enter your name"
required
/>
</div>

<!-- Insurance Type -->
```





Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



```
<div class="mb-4">
<label class="block font-medium mb-2"> Insurance Type </label>

<select
id="insuranceType"
class="w-full border rounded px-4 py-2"
required
>
<option value="">Select Insurance</option>

<option value="life">Life Insurance</option>

<option value="health">Health Insurance</option>

<option value="vehicle">Vehicle Insurance</option>
</select>
</div>

<!-- Coverage -->
<div class="mb-6">
<label class="block font-medium mb-2">
Coverage Amount (₹)
</label>

<input
type="number"
id="coverage"
class="w-full border rounded px-4 py-2"
placeholder="Example: 500000"
required
/>
</div>

<button
type="submit"
class="w-full bg-blue-600 text-white py-3 rounded hover:bg-blue-700"
>
Calculate Premium
</button>
</form>

<!-- Result -->
<div
id="result"
class="hidden mt-6 bg-green-50 border border-green-300 rounded p-5"
>
<h3 class="font-bold text-green-800">Premium Result</h3>

<p id="resultText" class="mt-2 text-gray-700"></p>
</div>
</section>
</div>
```





Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



```
</main>

<!-- Footer -->
<footer class="bg-gray-800 text-white text-center py-5">
<p>© 2026 SecureLife Insurance</p>

<p class="text-gray-400 text-sm">Static Website Demo - IIS + AWS S3</p>
</footer>

<!-- JavaScript -->
<script src="js/app.js"></script>
</body>
</html>
```

- And a folder named js with app.js file in it.

```
const form = document.getElementById("premiumForm");

form.addEventListener("submit", function (event) {
  event.preventDefault();

  const customerName =
document.getElementById("customerName").value;

  const insuranceType =
document.getElementById("insuranceType").value;

  const coverage =
Number(document.getElementById("coverage").value);

  let rate = 0;

  switch (insuranceType) {
    case "life":
      rate = 0.02;
      break;

    case "health":
      rate = 0.015;
      break;

    case "vehicle":
      rate = 0.03;
```





Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



```
break;
}

const premium = coverage * rate;

const result = document.getElementById("result");

const resultText = document.getElementById("resultText");

resultText.innerHTML = `
  Hello <strong>${customerName}</strong>!
  <br>
  Your selected insurance coverage is
  <strong>₹${coverage.toLocaleString()}</strong>.
  <br>
  Indicative Annual Premium:
  <strong>₹${premium.toLocaleString()}</strong>
`;

result.classList.remove("hidden");
});
```

Output locally

After filling and clicking Calculate Premium button it will show:

Premium Result

Hello **Ranjan**!
Your selected insurance coverage is **₹45,00,000**.
Indicative Annual Premium: **₹90,000**



Presented by
Ranjan Bhattacharya



Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



Step 2: Enable IIS (Skip if already enabled)

- In Windows we'll use IIS as the local web server.
- Enable IIS if it is not enabled
 - Press Windows + R
 - Type in: optionalfeatures and press Enter
 - Enable Internet Information Services.

Enable at least:

```
Internet Information Services
├── Web Management Tools
│   └── IIS Management Console
└── World Wide Web Services
    ├── Common HTTP Features
    │   ├── Default Document
    │   └── Static Content
```

- Windows will install IIS.
- Verify IIS: <http://localhost> You should see the IIS welcome page.
- If you are using Visual Studio for ASP.NET Core Most probably IIS is already enabled.

Step 3: Host the Website Locally Using IIS

- Press Windows + R and type in inetmgr or load Internet Information Services (IIS) Manager.
- Create IIS Website
- In IIS Manager go to Sites option and right click
- Select Add Website...
- Give name of website: InsuranceWebsite
- Define physical path: C:\Websites\InsuranceWebsite (Your website folder path)
- Go to Bindings... from right panel-Edit Site and use Add...
 - Type: http
 - IP Address: All assigned
 - Port: 8090 (Use unoccupied port)
 - Host name: define as you want (Optional)
- Press OK.
- In Manage Website section click on Start/Restart
- In Browse Website section click on Browse *:8090 (http)
- Or go to browser and use <http://localhost:8090> or run with URL in browser

Step 4: Now Deploy the SAME Website to AWS S3

- We will upload the same website to an S3 bucket.
- Here in this example S3 is acting as the static website hosting platform.
- Sign in to the AWS Management Console.
- Go to Amazon S3
- Select Create bucket
- Give Bucket name. Bucket names must be globally unique.
- For example: securelife-insurance-static-2026-hf
- AWS Region: Choose the region appropriate for your training environment, for example:
 - Asia Pacific (Mumbai) - ap-south-1





Cloud Computing & Dev-Ops



Transform Code into Cloud Solutions: Master AWS Today

- Important S3 Settings
 - Block Public Access settings: uncheck: Block all public access
 - Acknowledge warning displayed by Amazon
- Create the Bucket
- Upload Website clicking upload button
- Upload: index.html and then upload the folder: js
- Enable Static Website Hosting: go to properties
- Find Static website hosting by scrolling down
- Click Edit
- Select Enable for hosting type: Host a static website
- Define Index document: index.html (case sensitive).
- Save changes.

Step 5: Define S3 Public Access Policy

For the classic S3 static website endpoint, the objects need to be publicly readable. You can add a bucket policy such as:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
    }
  ]
}
```

Step 5: Get the S3 Website Endpoint

- After enabling static website hosting, AWS provides a website endpoint. The exact endpoint depends on your bucket and region.
- The S3 Website Endpoint is shown at the **bottom of the Properties page**
- You can click / Open that endpoint in your browser.
- Your website should appear.
- Test it.

☁️🌐📱 No Limits to What You Can Store, No Limits to What You Can Learn. 📱🌐☁️

